

Array Mutations in Legacy Svelte

Q28

STATE (LEGACY)

INTERVIEW QUESTION

Question. Why does mutating an array using methods like `.push()` fail to trigger a reactive update by default in legacy Svelte?

ANSWER. In Svelte 3 and 4, reactivity is driven strictly by variable assignment (`=`). Array methods like `.push()`, `.pop()`, or `.splice()` mutate the underlying array in memory without reassigning the variable reference itself. Because the Svelte compiler instruments assignments rather than observing memory mutations at runtime, the invalidation routine is never triggered unless the variable is explicitly reassigned.

App.svelte

```
<script>
  let items = ['Apple', 'Banana'];

  function addItem() {
    // ❌ Mutates array in-place, but does NOT reassign 'items'
    items.push('Cherry');

    // ✅ Legacy fix: Reassignment triggers Svelte's compile-
    // time invalidation
    items = items;
  }
</script>

<button on:click={addItem}>Add Item</button>
```

```
<ul>
  {#each items as item}
    <li>{item}</li>
  {/each}
</ul>
```

- ❗ **Summary.** The Svelte 3/4 compiler cannot track internal mutations to objects or arrays at runtime. You must always use the assignment operator to notify Svelte that a variable's state has changed and the DOM needs an update.

NO REASSIGNMENT (UI IGNORED)

`items.push('Cherry')`

X

ASSIGNMENT DETECTED (UI UPDATED)

`items = items`

✓